

Comprehensive **Ruby on Rails**

A step-by-step masterclass on Controllers, Views, the link_to Helper, Bootstrap 5
Integration, and ActiveRecord Models

Course Agenda

Module 1: Demystifying Controllers (Manual vs Generators)

Module 2: Mastering the `link_to` Helper

Module 3: DRYing up Views with Partial

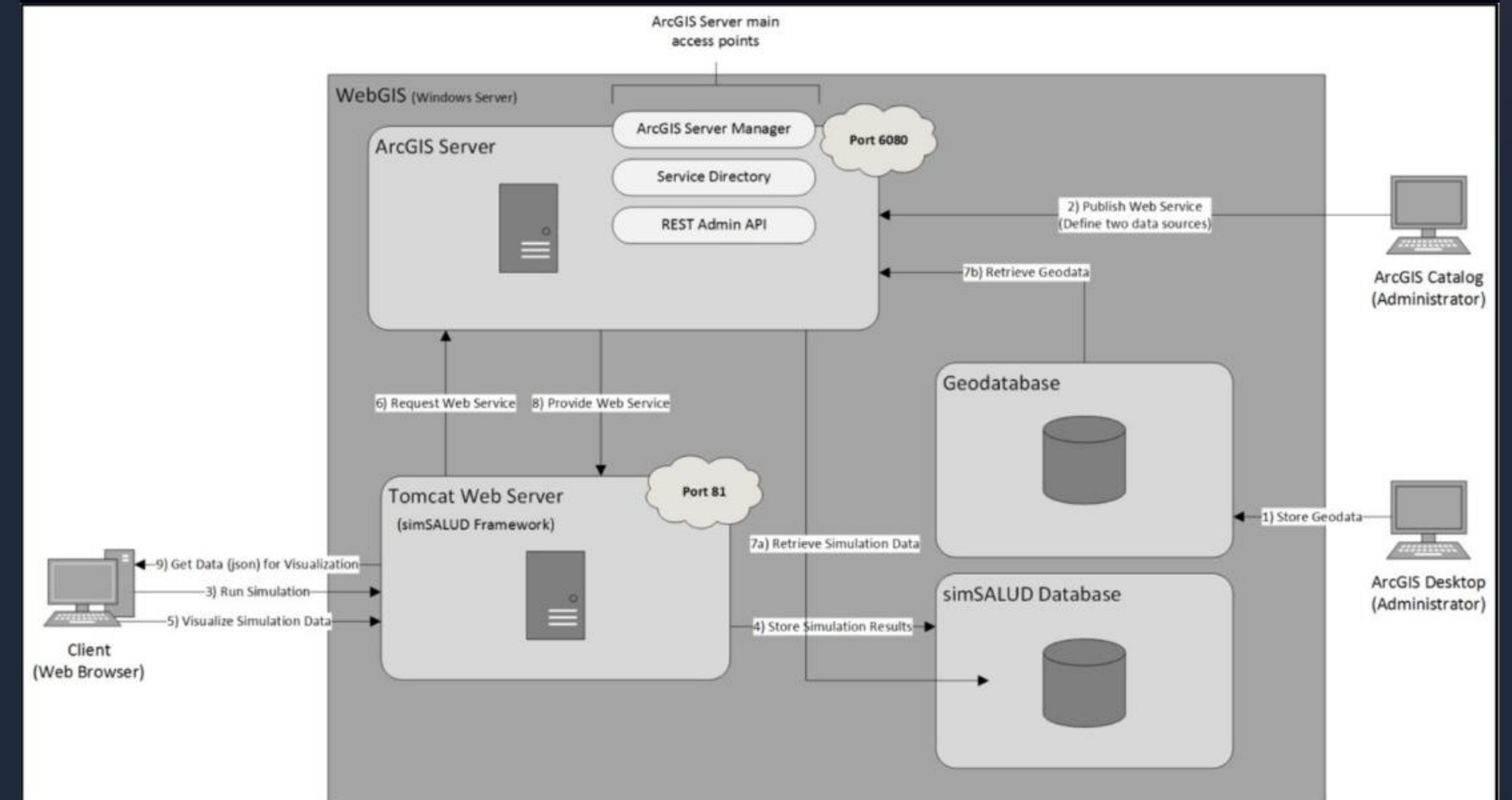
Module 4: Integrating Bootstrap 5 via CDN

Module 5: Generating your first ActiveRecord Model

What is a **Controller**?

Before creating one, we must understand its role. Rails uses the MVC (Model-View-Controller) architecture.

- The **Controller** is the "brain" or traffic cop of your application.
- It receives incoming HTTP requests from the browser (via the router).
- It asks the **Model** for data from the database.
- It passes that data to the **View** to generate the HTML sent back to the user.



Module 1

Creating Controllers: Manual vs. Generator

Method 1: **The Manual Approach**

Why do it manually?

Creating a controller entirely from scratch is the best way for beginners to understand exactly how Rails connects different files together without relying on "magic".

What is required?

To make a single page work manually, you must independently create three separate things:

- A Ruby controller file.
- A Route in your configuration.
- An HTML (ERB) View file.

Manual Step 1: Creating the File



Create the file: Navigate to `app/controllers/` and create a new file named `pages_controller.rb`. Notice the plural naming convention.



Define the Class: Open the file and write:

```
class PagesController < ApplicationController  
end
```



Add an Action: Inside the class, define a method. This is called an "action".

```
def home  
end
```

Manual Step 2: **Setting up Routes**



Open Routes: Go to `config/routes.rb`. This file tells your app where URLs should go.



Write the Route: Inside the block, add:

```
get 'home', to: 'pages#home'
```



What this means: When a user goes to `/home`, send the request to the `PagesController` and execute the `home` action.

Manual Step 3: **Creating the View**



Create a Folder: Go to `app/views/` and create a folder named exactly after your controller: `pages/`.



Create the File: Inside that folder, create a file named after your action with the `.html.erb` extension:
`home.html.erb`.

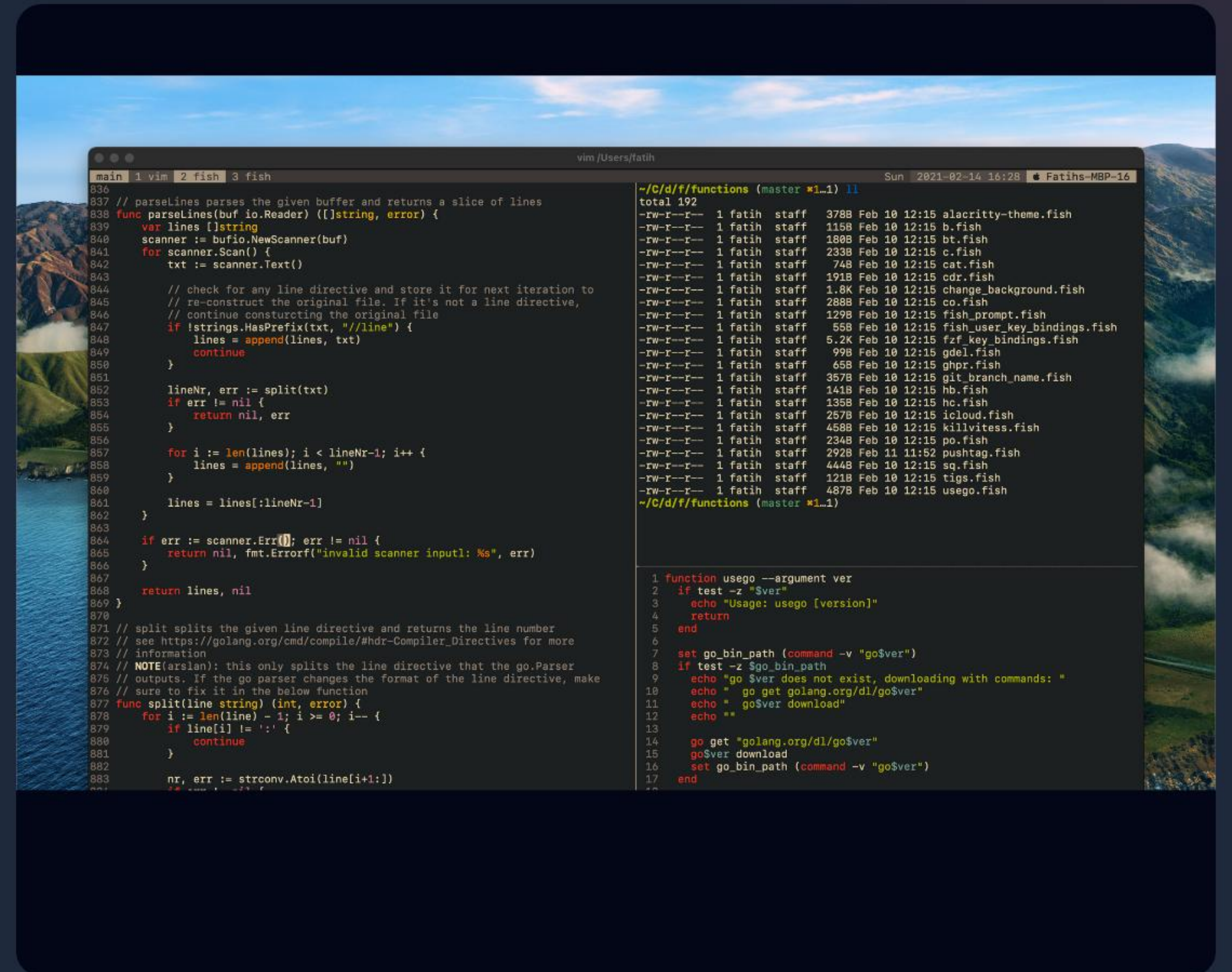


Add Content: Add some basic HTML like `Welcome Home` to test that it works.

Method 2: Using the Generator

The manual method took 3 distinct steps and modifying 3 different directories. Rails generators automate this entirely.

- Generators represent the "Rails Way," instantly scaffolding functional boilerplate code.
- They save significant development time.
- They prevent common errors like spelling mistakes in file names or mismatched route definitions.



The screenshot shows a terminal window with a dark background and a light blue border. The window is titled "vim /Users/fatih" and shows a Go file named "fish". The code is a function "parseLines" that takes a buffer and returns a slice of lines. It uses a scanner to parse the buffer and returns the lines. The code is commented with "NOTE(arslan): this only splits the line directive that the go.Parser outputs. If the go parser changes the format of the line directive, make sure to fix it in the below function". The code is split into two parts: "parseLines" and "split". The "split" function splits a line directive into a line number and a line. The "parseLines" function uses "split" to parse the buffer and returns the lines. The terminal also shows a file listing of the current directory, listing files like "alacrity-theme.fish", "b.fish", "bt.fish", "c.fish", "cat.fish", "cdr.fish", "change_background.fish", "co.fish", "fish_prompt.fish", "fish_user_key_bindings.fish", "fish_key_bindings.fish", "ghpr.fish", "git_branch_name.fish", "hb.fish", "hc.fish", "icloud.fish", "killvitess.fish", "po.fish", "pushtag.fish", "sq.fish", "tigs.fish", and "usego.fish".

```
main 1 vim 2 fish 3 fish
836
837 // parseLines parses the given buffer and returns a slice of lines
838 func parseLines(buf io.Reader) ([]string, error) {
839     var lines []string
840     scanner := bufio.NewScanner(buf)
841     for scanner.Scan() {
842         txt := scanner.Text()
843
844         // check for any line directive and store it for next iteration to
845         // re-construct the original file. If it's not a line directive,
846         // continue constructing the original file
847         if !strings.HasPrefix(txt, "//line") {
848             lines = append(lines, txt)
849             continue
850         }
851
852         lineNr, err := split(txt)
853         if err != nil {
854             return nil, err
855         }
856
857         for i := len(lines); i < lineNr-1; i++ {
858             lines = append(lines, "")
859         }
860
861         lines = lines[:lineNr-1]
862     }
863
864     if err := scanner.Err(); err != nil {
865         return nil, fmt.Errorf("invalid scanner input: %s", err)
866     }
867
868     return lines, nil
869 }
870
871 // split splits the given line directive and returns the line number
872 // see https://golang.org/cmd/compile/#hdr-Compiler_Directives for more
873 // information
874 // NOTE(arslan): this only splits the line directive that the go.Parser
875 // outputs. If the go parser changes the format of the line directive, make
876 // sure to fix it in the below function
877 func split(line string) (int, error) {
878     for i := len(line) - 1; i >= 0; i-- {
879         if line[i] != ':' {
880             continue
881         }
882     }
883     nr, err := strconv.Atoi(line[i+1:])
884     return nr, err
885 }
```

```
~/C/d/f/functions (master *1..1) ll
total 192
-rw-r--r-- 1 fatih staff 3788 Feb 10 12:15 alacrity-theme.fish
-rw-r--r-- 1 fatih staff 1158 Feb 10 12:15 b.fish
-rw-r--r-- 1 fatih staff 1808 Feb 10 12:15 bt.fish
-rw-r--r-- 1 fatih staff 2338 Feb 10 12:15 c.fish
-rw-r--r-- 1 fatih staff 748 Feb 10 12:15 cat.fish
-rw-r--r-- 1 fatih staff 1918 Feb 10 12:15 cdr.fish
-rw-r--r-- 1 fatih staff 1.8K Feb 10 12:15 change_background.fish
-rw-r--r-- 1 fatih staff 2888 Feb 10 12:15 co.fish
-rw-r--r-- 1 fatih staff 1298 Feb 10 12:15 fish_prompt.fish
-rw-r--r-- 1 fatih staff 558 Feb 10 12:15 fish_user_key_bindings.fish
-rw-r--r-- 1 fatih staff 5.2K Feb 10 12:15 fish_key_bindings.fish
-rw-r--r-- 1 fatih staff 998 Feb 10 12:15 gdel.fish
-rw-r--r-- 1 fatih staff 658 Feb 10 12:15 ghpr.fish
-rw-r--r-- 1 fatih staff 3578 Feb 10 12:15 git_branch_name.fish
-rw-r--r-- 1 fatih staff 1418 Feb 10 12:15 hb.fish
-rw-r--r-- 1 fatih staff 1358 Feb 10 12:15 hc.fish
-rw-r--r-- 1 fatih staff 2578 Feb 10 12:15 icloud.fish
-rw-r--r-- 1 fatih staff 4588 Feb 10 12:15 killvitess.fish
-rw-r--r-- 1 fatih staff 2348 Feb 10 12:15 po.fish
-rw-r--r-- 1 fatih staff 2928 Feb 11 11:52 pushtag.fish
-rw-r--r-- 1 fatih staff 4448 Feb 10 12:15 sq.fish
-rw-r--r-- 1 fatih staff 1218 Feb 10 12:15 tigs.fish
-rw-r--r-- 1 fatih staff 4878 Feb 10 12:15 usego.fish
~/C/d/f/functions (master *1..1)
```

```
1 function usego --argument ver
2 if test -z "$ver"
3     echo "Usage: usego [version]"
4     return
5 end
6
7 set go_bin_path (command -v "go$ver")
8 if test -z $go_bin_path
9     echo "go $ver does not exist, downloading with commands: "
10    echo "  go get golang.org/dl/go$ver"
11    echo "  go$ver download"
12    echo ""
13
14    go get "golang.org/dl/go$ver"
15    go$ver download
16    set go_bin_path (command -v "go$ver")
17 end
```

The Generator Command

The Command Line

Open your terminal and type the following command:

```
rails generate controller Pages home about
```

Or use the shorthand:

```
rails g controller Pages home about
```

Breaking it down

- `rails g controller` : Tells Rails we want to generate a new controller.
- `Pages` : The name of the controller (capitalized and plural).
- `home about` : The specific actions (methods) we want inside the controller.

What the Generator **Actually** Does

- ✓ **Creates Controller:** Generates `app/controllers/pages_controller.rb` with empty `home` and `about` methods pre-written.
- ✓ **Configures Routes:** Automatically adds `get 'pages/home'` and `get 'pages/about'` to your `routes.rb` file.
- ✓ **Creates Views:** Generates the `app/views/pages/` folder and creates both `home.html.erb` and `about.html.erb` inside it.
- ✓ **Creates Tests:** Scaffolds test files for you automatically.

Manual vs. Generator Comparison

Feature	Manual Creation	Rails Generator
Speed	Slow, requires multiple steps	Instant, one command
Error Prone	High (typos in file names/routes)	Low (strictly follows conventions)
Learning Value	Excellent for absolute beginners	Best for experienced developers
Test Scaffolding	Must be done manually	Automatic

Module 2

Working with the link_to Helper

What are **View Helpers**?

-  Rails provides "View Helpers" which are Ruby methods that generate HTML for you automatically.
-  Using helpers keeps your views clean, dynamic, and closely tied to your application's routing logic.
-  `link_to` is the most commonly used view helper in Rails. It generates standard HTML  anchor tags.

Basic Syntax of `link_to`

The Structure

The basic syntax requires two main arguments:

```
<%= link_to "Display Text", target_url %>
```

- **Argument 1:** The text the user will click on.
- **Argument 2:** The destination URL or routing path.

The ERB Tags

Notice the `<%=` and `%>` tags?

These are ERB (Embedded Ruby) tags. The equal sign (`=`) tells Rails to output the result of the Ruby code directly into the HTML.

Without the `=`, the ruby code would run silently in the background.

Replacing HTML tags

Why replace hardcoded HTML? Hardcoding URLs is brittle. If you change a route later, you must find and update every single HTML link manually.

Standard HTML:

About Us

Rails link_to equivalent:

```
<%= link_to "About Us", "/about" %>
```



Step 1: Using **Named Routes**



Instead of hardcoding `"/about"`, Rails gives route paths a specific name.



You can see all your named routes by typing `rails routes` in your terminal.



For example, the `/about` route is usually named `about_path`.



The proper Rails way:

```
<%= link_to "About Us", about_path %>
```

Step 2: Linking to Models



If you want to link to a specific item from your database (like a specific blog post or user profile), `link_to` makes it incredibly easy.



You do not need to manually construct the URL like `"/posts/5"`.



Instead, just pass the Ruby object directly to the helper:

```
<%= link_to "Read Article", @article %>
```



Rails automatically figures out that it needs to go to the `show` action for that specific article's ID.

Passing CSS and **Attributes**



CSS Classes

Pass a hash of options to apply CSS formatting directly.

```
<%= link_to "Home",  
root_path, class: "btn btn-  
primary" %>
```



HTML IDs

Need to target a link for JavaScript? Pass an ID.

```
<%= link_to "Click", "#",  
id: "trigger-btn" %>
```



Target Blank

Easily open links in new tabs using standard target attributes.

```
<%= link_to "Google",  
"http://google.com",  
target: "_blank" %>
```

Advanced: Deletes & Confirmations

HTTP DELETE Method

HTML anchor links can only perform GET requests. To trigger a DELETE request (like destroying a record), use Hotwire/Turbo attributes.

```
<%= link_to "Delete", @post, data: {  
turbo_method: :delete } %>
```

Browser Confirmations

Before executing destructive actions, it's best practice to add a browser confirmation prompt to ensure user intent.

```
<%= link_to "Delete", @post, data: {  
turbo_method: :delete, turbo_confirm: "Are  
you sure?" } %>
```

Module 3

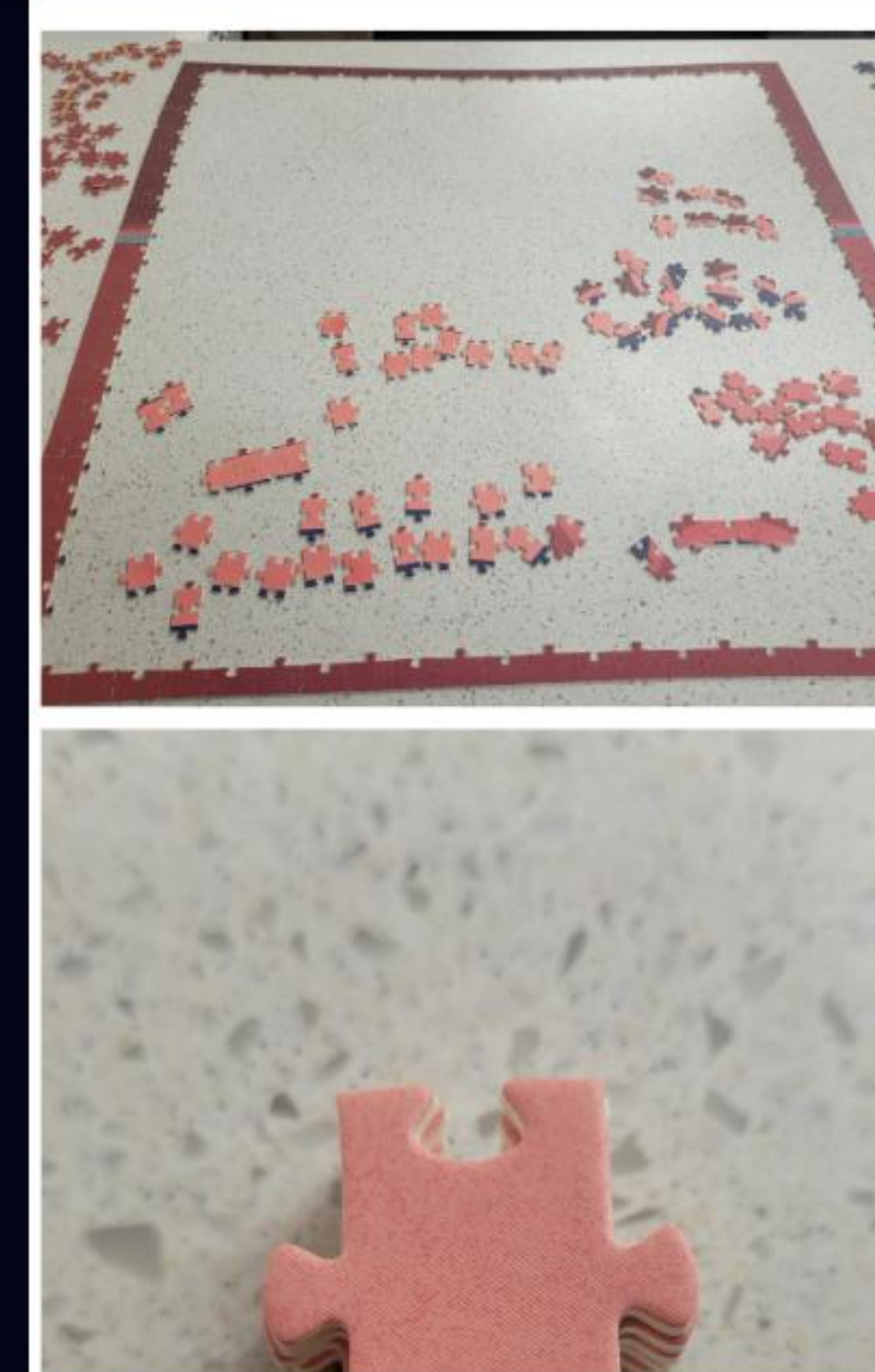
Using Partials to Refactor Views

The Problem of Duplication

As your application grows, you will notice sections of HTML repeating across different view files. The most common example is a form.

The form to "Create a New User" and the form to "Edit an Existing User" are usually 99% identical.

If you maintain two separate files, and decide to add a new "Last Name" field, you must update the code in two different places. This violates the DRY (Don't Repeat Yourself) principle.



What exactly is a **Partial**?



Partials are modular building blocks that allow you to break complex HTML rendering into small, manageable chunks.



Think of them as "mini view files" that you can inject into larger, main view files.



Naming Convention: Partial file names must *always* begin with an underscore. This tells Rails "this is not a full page, it is a partial."



Example: `_form.html.erb`

Identifying **Partial Candidates**



Navigation Bars

Headers, footers, and sidebars appear on every page. Extract them into `_navbar.html.erb`.

Data Forms

Create and Edit views share identical form inputs. Extract them into `_form.html.erb`.



Cards / Lists

Displaying a user profile card in a list vs on a dashboard? Extract it into `_user_card.html.erb`.

Step 1: Creating the File



Open your `new.html.erb` file where your form is currently written.



Cut (remove) all the HTML related to the form itself.



In the same directory (e.g., `app/views/users/`), create a new file named `_form.html.erb`.



Paste the HTML you just cut into this new partial file and save it.

Step 2: Rendering the Partial



Go back to your original `new.html.erb` file.



Use the `render` method to inject the partial where the form used to be.



Type: `<%= render "form" %>`



Notice that when you call `render`, you **drop the underscore** from the name!



Now go to `edit.html.erb` and use that exact same `render "form"` line. Both pages now share one form.

Step 3: The Danger of **Instance Variables**

The Mistake

If your partial relies on an instance variable from the controller (like `@user`), it becomes tightly coupled. It will crash if you try to render it from a controller action that doesn't define `@user`.

The Solution

Partials should be "dumb" and isolated components. They should not know about global instance variables. Instead, we must pass data into them explicitly as **Local Variables**.

Step 4: Passing Local Variables

Injecting Data

When calling `render`, pass a hash to define local variables.

Syntax:

```
<%= render "form", specific_user: @user %>
```

This takes the `@user` instance variable, and hands it to the partial under the local name `specific_user`.

Using the Data

Inside `_form.html.erb`, you no longer use the `@` symbol. You simply use the local variable name you defined.

```
<%= form_with model: specific_user do  
  |form| %>
```

Now, your partial is completely reusable!

Module 4

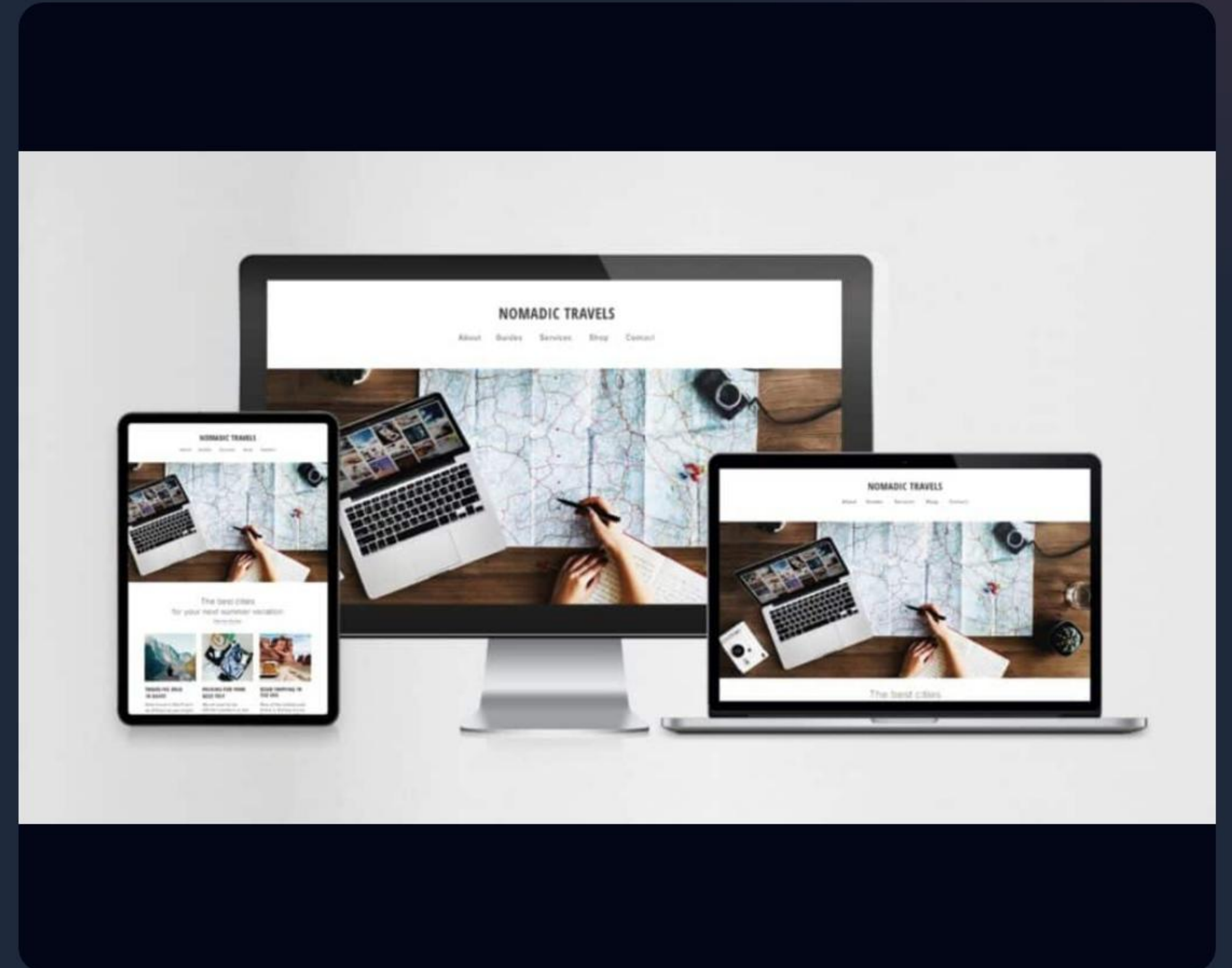
Adding Bootstrap 5 via CDN

Why Bootstrap via **CDN**?

Adding a complete CSS framework manually through asset bundlers (like Webpack or esbuild) can be highly complex and frustrating for beginners.

A Content Delivery Network (CDN) allows you to link directly to Bootstrap's hosted files on the internet.

This is by far the fastest approach to get a responsive, modern UI running in Rails without touching your asset pipeline.



Step 1: The **Application Layout**



In Rails, you do not need to add the same `` tags to every single page.



Rails uses a master layout file that wraps around all your individual view templates.



Locate and open: `app/views/layouts/application.html.erb`.



This file contains your main `<html>`, `<body>`, and `<div>` tags.

Step 2: Linking the **CSS**



Go to the official Bootstrap 5 documentation site and find the "Include via CDN" section.



Copy the provided `<link>` tag.



Paste this link directly inside the `<head>` section of your `application.html.erb` file.



Best practice: Place it *above* your custom stylesheet links so you can override Bootstrap classes if needed later.

Step 3: Linking the JS Bundle

-  Bootstrap CSS handles colors and grids, but components like Dropdowns, Modals, and mobile Navbars require JavaScript.
-  Copy the `</div>` bundle tag from the Bootstrap website.
-  Paste this script tag right before the closing `</body>` tag in your layout file.
-  Putting it at the bottom ensures the HTML loads fully before the JavaScript executes, speeding up page load times.

Step 4: Testing the Setup

Applying Classes

Go to any of your standard ERB view files (e.g. `home.html.erb`).

Wrap your content in a container and add a button:



Test



Verifying

Start your Rails server (`rails s`) and navigate to the page.

If you see a nicely styled, red button pushed down slightly from the top margin, your CDN installation is successful and globally available!

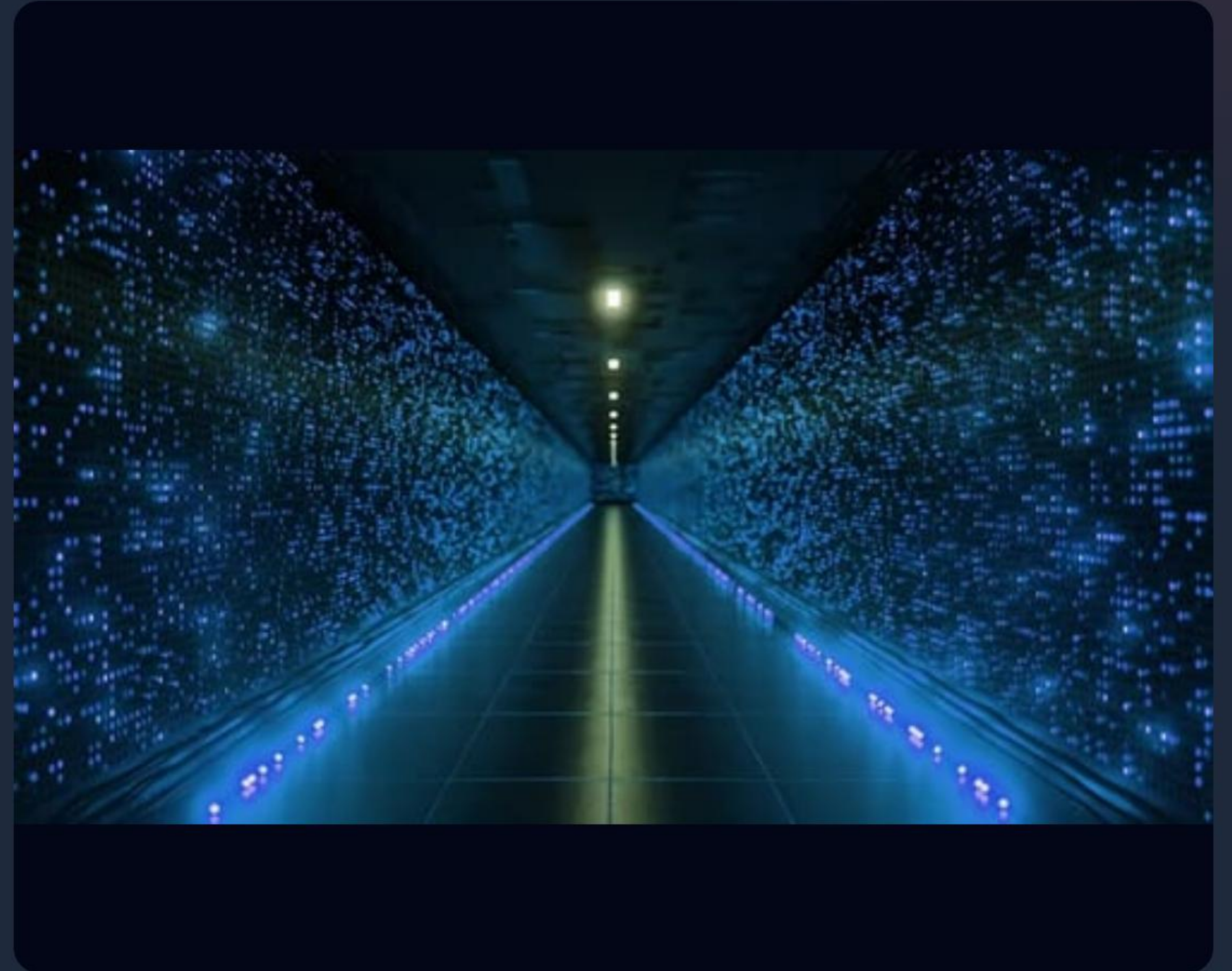
Module 5

Generating an ActiveRecord Model

What is a **Model**?

In the MVC framework, the **Model** is the 'M'. It represents the data layer.

- Rails uses a system called **ActiveRecord**.
- ActiveRecord acts as an ORM (Object-Relational Mapping) system.
- It translates complex SQL database queries into simple, readable Ruby code.
- Generating a model creates the Ruby class AND the instructions to build the actual database table.



Step 1: The Generation **Command**

 Open your terminal. We will use another Rails Generator.

 Type the following command exactly:

```
rails generate model Article title:string body:text
```

 **Crucial Naming Rule:** Model names must be *Singular* and *Capitalized* (e.g., `Article`, not `articles`).

 After the name, you list the attributes (columns) you want the database table to have, followed by their data type.

Understanding **Data Types**

A large, stylized red letter 'A' icon.

:string

Used for short text, usually limited to 255 characters. Perfect for Titles, Names, and Email addresses.

A red icon consisting of three horizontal lines, representing a list or text block.

:text

Used for long-form data without a strict character limit. Perfect for Blog Bodies, Descriptions, and Comments.

A large, stylized red hash symbol '#' icon.

:integer

Used for whole numbers. Perfect for Age, Quantity, or View Counts. Other types include boolean, datetime, and float.

Step 2: Inspecting the Migration

-  Running the generator creates a Model file (`app/models/article.rb`) and a **Migration file**.
-  Navigate to `db/migrate/` and open the newly created file (it will have a long timestamp in the name).
-  Look at the code. It is a set of Ruby instructions telling the database engine (like SQLite or PostgreSQL) exactly how to create a table named `articles` with `title` and `body` columns.
-  Note: The table has not been created yet! The migration is just the blueprint.

Step 3: Running `db:migrate`

Executing the Blueprint

To finalize the process, you must tell Rails to execute the migration blueprint against your actual database.

Run Command:

```
rails db:migrate
```

The Result

Your database table now officially exists.

Rails will also generate an `id` column automatically, along with `created_at` and `updated_at` timestamps.

You can now open the Rails console (`rails c`) and start saving `Article` data!

Questions?

Thank you for your attention.

Ruby on Rails - Comprehensive Masterclass

Image Sources



<https://microsimulation.pub:8081/iiif/2/00159%2Fijm-00159-fig5.tif/full/1500,/0/default.jpg>

Source: www.microsimulation.pub



<https://arslan.io/content/images/2022/09/image-217.png>

Source: arslan.io



https://plus.unsplash.com/premium_photo-1764705739194-25b4f8de4d5b?fm=jpg&q=60&w=3000&auto=format&fit=crop&ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1yZWxhdGVkfDE1fHx8ZW58MHx8fHx8

Source: unsplash.com



<https://preview.redd.it/my-puzzle-is-full-of-nearly-identical-pieces-that-all-fit-v0-5j49za9vzyfg1.jpeg?auto=webp&s=3598f02ebca6da924d6bac85fda87c01edeac73b>

Source: www.reddit.com



<https://www.skillshare.com/blog/wp-content/uploads/2024/04/skillshare-responsive-web-design-101-1024x574.jpg>

Source: www.skillshare.com



https://elements-resized.envatousercontent.com/elements-video-cover-images/ea7c0419-922a-4265-9766-a35d580ee927/video_preview/video_preview_0000.jpg?w=500&cf_fit=cover&q=85&format=auto&s=062b6db53d370e8b7ccaa849ee580506cc47eb122d60b6594372ccb20d24f7e1

Source: elements.envato.com